



Recursive Self-Improving Data Production

对抗递归进化：由求解、验证、改环境三个 agent 组成的多智能体系统，持续生产高难度、不可作弊的终端任务与轨迹

ADVERSARIAL RECURSIVE EVOLUTION

THREE-AGENT MAS

PARAMETRIC TASK INSTANCES

VERIFIER IS THE ONLY REWARD

能力的瓶颈，已经从模型转移到题目

能力从哪里来

终端 agent 要学的不是知识，是**行为**：在信息不全的环境里试探、判断、纠错、收尾。这类行为只出现在“真的把一件事做完”的过程里，也只能从这样的过程中学到。而模型越强，还能提供梯度的题越少——能力的边界越来越取决于我们造得出多难的题。

难在自洽，不在数量

一道题不是一段文字，而是一个自洽的小世界：目标说得清、环境立得住、确实存在一条走得通的路、而且有人能客观判定走通了没有。人来写，一道题的成本高到无法规模化；交给模型批量生成，这四者很容易各说各话。最常见的失效不是“太难”，而是**“看起来合格”**。

"合格"本身有漏洞

今天判定一道题合格的方式是：存在一个正确解，且验收脚本认它。这证明了**有路可走**，没证明**必须走这条路**。若照抄答案、伪造产物也能通过，这道题给出的分数就不再代表能力；用它训练，模型会朝“让验收通过”进化，而不是朝“把事做成”进化。

所以要提高的不是题目的长度或数量，而是**“合格”这个标准本身的强度**：不仅要存在正确解，还要在给定预算内找不到廉价解。难度也随之换了定义——不是标准解有多长，而是**最短的通过解有多长**。

三个 agent，一个 verifier

Solving Agent

在沙盒里做任务，产生轨迹。它有两种模式：**正常模式**尽力解题，成功轨迹成为训练数据和后续的“证人”；**攻击模式**只给 1 到 8 条命令的预算、只看指令与环境，用最短最懒的方式试图通过 verifier。攻击模式的每一次成功，就是一个反例。

Verify Agent

唯一的裁决者。在多个环境实例上执行标准解并运行 verifier，给出可解性证书；回放攻击模式的轨迹，判定它是反例还是合法快解；检查修复后的 verifier 是否仍让所有证人通过。它输出逐项 check 结果，不输出意见。

Environment Agent

唯一能改任务的角色。提出更难的任务；收到反例后写出一条新 check，使反例失败而标准解与证人通过；把写死的 fixture 改成按 seed 生成的参数化实例，让常数不再是常数。

分工的原则：Solving Agent 只产生行为，Verify Agent 只裁决，Environment Agent 只改环境。LLM 的意见永远不进入 reward、loss mask 或评测分，reward 只来自 verifier。

对抗递归进化怎么做

1提出	Environment Agent	在 seed 任务上加一个新的终端子目标，产出指令、参数化环境、标准解、verifier。
2证书	Verify Agent	在 n 个环境实例上运行标准解并跑 verifier。全部通过才算"有通解"，只在一个实例上通过不算。
3攻击	Solving Agent · 攻击模式	在一个新实例上，用预算 B 条命令、只看指令与环境，尝试通过 verifier。预算从 1 逐级放大到 8，记录首次得手的预算。
4修复	Environment Agent	对每个反例写一条分离 check：反例失败，标准解和所有证人通过。写得出来，说明 verifier 有缺口；写不出来，说明反例是合法解，任务只是容易，记低难度而不修。
5接受	Verify Agent	预算内再找不到反例即接受，预算耗尽即拒绝。接受的任务带一个新标签：首次得手预算，它就是任务的难度。
6训练	Solving Agent · 正常模式	在存活任务上 rollout、筛选、训练；成功轨迹回流为证人，失败画像告诉 Environment Agent 先加固哪一类 check。

$\text{accept}(\tau) \Leftrightarrow \forall \theta \in \text{实例样本: 标准解通过} \vee \theta \mid \wedge \forall \xi \in \text{攻击类 } A_B: \xi \text{ 在新实例上不通过} \mid \wedge \forall w \in \text{证人集: } w \text{ 在新实例上通过}$
难度(τ) = 最短通过解的命令数，而不是标准解的长度。

起点：一道已经"验收合格"的任务

任务指令（节选）

```
用 ~/keys 里的私钥 SSH 到 10.188.74.102，运行
migrationConfigGenerator.py --cluster cl1 --host
c7401...，
输出重定向到 /app/result.txt。
在 /etc 下找到 required_services.txt，把生成结果里出现
而清单里缺少的服务追加进去。
对清单里每个服务，确认它出现在 result.txt 中，
把 "<服务名> PASS" 逐行写进 /app/validation.txt。
```

环境 v0：写死的 fixture

```
printf 'ZOOKEEPER\nAMBARI_INFRA_SOLR\nRANGER\nATLAS\n' \
> /etc/service-lists/required_services.txt
# 远端生成器的输出固定多出一个 LOGSEARCH
# 标准解里也写死了：grep -qF "LOGSEARCH" result.txt
```

verifier v0 的四项 check

- 01 清单文件含 LOGSEARCH，且 mtime 晚于 result.txt
- 02 result.txt 非空，含三个固定字符串：开始标记、主机名、结束标记
- 03 清单里每个服务在 validation.txt 里有一行 X PASS，且无 FAIL
- 04 "反捷径"：validation.txt 不是清单的逐字复制；mtime 晚于 result.txt；再查一次三个标记

标准解跑通，四项全绿，这道题进了任务池。但 v0 检查的全是**常数和时间戳**：三个固定字符串、一个写死的服务名、两个 mtime 先后。标准解和 verifier 都只对这一个实例成立。下一页看它如何被两个反例推着进化。

两个反例，两次进化，然后存活

ROUND 0 → 1·驱动进化的反例：换一个实例

环境 v0

缺失服务固定为 LOGSEARCH；verifier 找固定字符串；标准解写死 LOGSEARCH。

反例 A·Verify Agent 生成新实例 θ_2

远端生成器这次多出的是 KAFKA。标准解仍追加 LOGSEARCH，check 01 仍要求 LOGSEARCH：**标准解与 verifier 同时失效**。这是“只对一个实例可解”的证据。

Environment Agent 的修复 → v1

缺失服务集合与主机名按 seed 生成；标准解改为集合差 `结果 - 清单`；verifier 验证时自己拉取本实例生成器的真实输出，逐行比对 result.txt，不再找固定字符串。

ROUND 1 → 2·驱动进化的反例：跳过校验

环境 v1

result.txt 须等于真实输出；清单 $\supseteq$ 结果中的服务；validation 对清单里每个服务有 PASS。

反例 B·Solving Agent 攻击模式，预算 3

```
ssh ... > /app/result.txt
grep -oE '[A-Z_]+' /app/result.txt >> 清单
sed 's/$/ PASS/' 清单 > /app/validation.txt
```

第三条命令给**所有**服务写 PASS，从未检查它是否真在结果里。v1 分不清“校验过”与“假定过”。

Environment Agent 的修复 → v2

清单初值里按 seed 放一个**诱饵服务**（结果里不存在）；指令改为逐行写 PASS 或 FAIL；verifier 从两个文件重算 $\text{PASS 集} = \text{清单} \cap \text{结果}$ 、 $\text{FAIL 集} = \text{清单} - \text{结果}$ 。反例 B 对诱饵写了 PASS，失败。证人检查：通解版标准解与两条真正逐项校验过的学生轨迹仍通过。

ROUND 2·存活

环境 v2

参数化实例、真实输出比对、诱饵、交叉重算。

攻击预算放大到 8，无反例

伪造 result.txt 过不了真实输出比对；不读结果写不出诱饵的 FAIL；复制清单过不了交叉重算。最短通过解必须含 SSH、集合差与逐项校验。

接受，难度标签“预算 8 存活”

Round 0 的最短通过解是 4 条命令、零次 SSH；Round 2 已等于任务本身。任务没有变长，变的是**最短捷径的长度**。

Round 0 来自任务池中的真实任务及其 verifier 源码；反例 A、B 与修复 v1、v2 是对该 verifier 的推演，尚未在沙盒中执行。

为什么要这么做

两个缺口相加：一道"合格"的题可以不迫使求解者读实例，而模型已经学会提前收工

任务侧·验收标准留下的缺口：不重算、不换实例，答案就是常数

NO RECOMPUTE NEEDED

23.9%

不必从实例重算就能满足的检查：只查存在性 9.2% + 只比常数 2.4% + 读产物再比常数 12.3%。它们对"复制标准解的产物"没有抵抗力。对照：67.2% 确实在重算或重跑。

SINGLE-INSTANCE TASKS

85.7%

只有 14.3% 的任务在构建时引入随机性。其余答案是常数，写死在原理上不可检出——参数化实例最直接的理由。

VERIFIER READS THE ORACLE

599

去读标准解脚本找硬编码值的 verifier 数；其中 249 个断言该脚本存在，而 `/solution` 只在跑 oracle 时挂载，这些任务对任何 agent 都不可解。

MUTATION-READY

64.1%

24,028 个任务有"Dockerfile 烤进去、被读、标准解不写"的 fixture，可自动改一个数据 token。换实例实验的可行上界。

模型侧·失败不是做不完，是做了六成就宣布做完

FAILURES THAT CLAIM DONE

91.9%

失败轨迹里最后一步宣称完成的比例；撞到轮数上限的是 0%。失败是提前收工，不是跑不完。

PARTIAL CREDIT AT FAILURE

0.60

失败时已通过的 check 占比中位数。模型做了六成就宣布完成。

DID IT, TRIPPED PROCESS

35%

带编号 check 的 712 条失败中，最终语义已过、却挂在过程或反捷径 check 上的比例。是抓到捷径还是过度特化，只有证人能分辨。

SUCCESS LENGTH VS ORACLE

9 → 14

标准解从 50 行长到 400 行，成功轨迹的中位轮数只从 9 到 14。任务在变长，不在变难。

上排：全部 37,484 个任务、309,712 个测试函数，规则版本 2026-09-04 (`scripts/22_probe_verifiers.py`，两套口径均存于 manifest)。下排前三格：`trajectories-000000.tar` 的 4,429 条干净轨迹，全部 Qwen3.5-27B；第四格为全量 60,932 条成功轨迹。轨迹侧尚未全量复核。

生产出来的一条优质 agent 数据

T1 先看密钥是否存在，再建立连接并运行生成器。

```
ls -la ~/keys/ · chmod 600 ~/keys/insecure_private_key · ssh -i ...  
root@10.188.74.102 'migrationConfigGenerator.py ...' > result.txt
```

T2 SSH 成功。查看输出内容，并定位清单文件。

```
cat /app/result.txt · find /etc -name 'required_services.txt'
```

T3 结果里检测到 5 个服务，其中 LOGSEARCH 不在清单里。

```
cat /etc/service-lists/required_services.txt
```

T4 追加缺失服务，逐项校验并写报告。

```
echo LOGSEARCH >> required_services.txt · while read s; do grep -q "Service  
detected: $s" result.txt && echo "$s PASS"; done > validation.txt
```

T5 validation.txt 是空的，grep 模式可能没匹配上。手动测试模式，逐个服务检查。

```
grep 'Service detected:' result.txt · ... grep -q "$s" result.txt && echo  
Found
```

T6 模式修正后 5 个服务全部 Found。重写报告，并核对两个文件都非空。

```
while read s; do grep -q "$s" result.txt && echo "$s PASS" >>  
validation.txt; done · ls -la result.txt validation.txt
```

T7 逐条对照指令确认五项要求均已满足，宣称完成。 `task_complete: true`

VERIFIER

4 / 4

四项 check 全部通过

COMMANDS

17

7 个回合，15 条不重复

探索

先 ls、cat、find 再动手，缺失服务是从输出里读出来的，不是猜的。

恢复

T4 写出空文件，T5 定位到 grep 模式问题，T6 修正。一次真实的错误诊断与恢复。

自检

宣称完成前用 ls -la 核对产物，再逐条对照指令。

干净

从未触碰 tests/ 或 verifier；保留原始 completion 与逐项 check 结果，可直接重构为训练样本。

来源：任务池同一任务上 27B 求解模型的一条成功轨迹，8 个回合，1,696 个输出 token，reward 1.0。文字为模型 analysis 字段的节译。

Bottom Line

对抗递归进化：每一轮，求解者攻击环境，环境在验证者的裁决下进化，存活的任务才进入训练。标准解通过只是入场券；还要在预算内找不到捷径、此前的合法解仍然通过、换一个实例后答案不再是常数。三个 agent 各管一段，verifier 始终是唯一的重赏。

NOT CHEAPLY SOLVABLE

SOLVE · VERIFY · REPAIR

PARAMETRIC INSTANCES

VERIFIER-ONLY REWARD